Разложить число на множители методом Ферма

n[1]= 459815140061495110658001985470439569453709795016952965385917
n[2]= 10762594122318150198651258041676055928393157424115409732728 39
n[3]= 14554137880788423915514734980557110302769737399620545126091
n[4]= 14787289758899679868141151766853045415170081745809047130 6057
n[5]= 34330706771161504737218296235662944295228926710887500395620 1
n[6]= 11391035927007347815972448788654755662185624388776525399183
n[7]= 17708900652112934265497878309058883039113528211411174267036 7
n[8]= 18348390354366311880068333550006043879737980180139631920049 1
n[9]= 50960820726467380475310200481526194379804558074716505877991 3
n[10]= 15866848316921316101532963207975611382175675043882547804 59
n[11]= 96475838745735468360998004546471216075336887487636131901823 9
n[12]= 92978831446527927064539719775506181151010499772447865126477
n[13]= 21078917190183585592715638706004862806827590109994148804973
n[14]= 10331135123062901353510888721709975580692939762510902263478 9
n[15]= 13376963399289829095866288084388746663914491186238964546930 47
n[16]= 10898105077927627203622548301412367455767121377082027692542 43
n[17]= 11908579022330552392382779117555763691024248936504508614517 7
n[18]= 54182026308551218369283079814951538336114772317465911618851 1
n[19]= 43742084152204964329128935327697851564591940767363981591 11
n[20]= 13533541753285984782454691151010712305885840641029134576667 93
n[21]= 13066918110572271028962668215635646397950653734048586049039 19
n[22]= 31749567681059905723170904782339155022184512900904008862126 7
n[23]= 49951794106007045631366557654703900333629202038723682894445 9
n[24]= 50082223848612603785863008899053942431257791369989299663652 1
n[25]= 18508076007695806953980177881350121410831918966865563895456 7
n[26]= 33782695704184206815309839169982180278293407644988644744 59
n[27]= 11780161793437394703940393836282255911990124382734239941228 73
n[28]= 40281903763129181504376244655630311991984759604652279109549 3
n[29]= 55363573859284102401503605223617234914115530100672766725264 3
n[30]= 90531694464813459650597725911504303945211895831619408158089

В 1643 году Пьер де Ферма предложил метод факторизации. Этот метод позволяет эффективно раскладывать целые числа на простые множители.

Что такое метод факторизации Ферма?

Идея метода состоит в том, что произведение двух простых чисел можно представить в виде N=(A-B)(A+B), где A — это среднее арифметическое двух простых чисел (A=(p+q)/2), а B — это расстояние от A до искомых простых чисел (B=p-a=a-q).

Если простые числа находятся близко друг к другу, то и A близко к квадратному корню из N. Это позволяет определить значение A путем перебора: начать с квадратного корня из N и увеличивать потенциальное значение A на единицу в каждой итерации.

Для каждой итерации мы можем вычислить B^2 = A^2 — N. Если результат является квадратом, то мы угадали A. И теперь можем вычислить p=A+B и q = A-B.

```
def perfectSquare(x):
    return (int(x**0.5))**2 == x:

def factorizeFerma(n):
    x = int(n ** 0.5) + 1
    while not perfectSquare(x * x - n):
        x += 1
    y = int((x * x - n) ** 0.5)
    a = x - y
```

```
    b = x + y
    return a, b

n = int(input())
a, b = factorizeFerma(n)
print(a, b)
```

n=pq,

p≈q=> n≈p^2

k=окр(корень(n)) => пока n/k не целое выполнять k+=2

Первоначально Ферма описал этот метод в своём письме, датированным 1643 годом. Текст оригинального письма можно найти в Oeuvres de Fermat, на 256 странице.

Ключ RSA уязвим, если два простых числа p и q находятся близко. Если простые числа генерируются независимо друг от друга и случайным образом, то вероятность того, что они будут близки, ничтожно мала.

Однако функции генерации ключей RSA могут реализовывать ошибочный алгоритм, например такой:

Сгенерировать случайное простое число X;

Найти следующее простое число и присвоить его q;

Найти следующее простое число и присвоить его p;

Для стандартных размеров ключей RSA разница p и q — тысячи или меньше. Такой алгоритм является уязвимым для метода факторизации Ферма.

Какой должна быть разница простых чисел, чтобы ключ RSA был уязвим?

При обычных размерах ключей RSA в 2048 бит метод ферма со 100 итерациями надежно факторизует числа, где разница p и q меньше 2^517. Другими словами, простые числа, различающиеся только младшими 64 байтами, будут уязвимы. Можно было бы возразить, что 100 итераций — это слишком много, однако алгоритм настолько быстр, что на практике это не будет иметь большого значения.

Могут ли случайно сгенерированные ключи быть уязвимы?

Да, могут. Но для этого необходимо, чтобы они были идентичны, по меньшей мере, в своих старших 500 битах. Вероятность такого исхода — 1:2^500.

Метод работает быстро, если n является произведением двух близких к друг другу сомножителей. В частности, именно поэтому в алгоритме RSA требуют, чтобы разность между секретными простыми сомножителями модуля была велика.
Поскольку факторизация чисел является актуальной задачей (в частности, для криптографии), математиками разработано большое количество алгоритмов, которые с высокой эффективностью за ограниченное время позвляют разложить число на множители: *P-1 метод Полларда, P-алгоритм Полларда, Метод Лемана, Метод эллиптических кривых (алгоритм Ленстры), Алгоритм Диксона, Квадратичное решето* и другие.